ARTIFICIAL INTELLIGENCE - LAB PROJECT 2025/2026

The objective of this analysis is to predict the winner of an ATP tennis match using historical data. The provided datasets encompass match records from 2015 to 2025, capturing comprehensive details about tournament conditions and player statistics.

Data framing and preparation

We use the ATP match-level dataset from tennis-data.co.uk, covering ten seasons (2015-2025). Each row contains match metadata (tournament, surface, round, date), player identifiers and rankings (WRank, LRank), and bookmaker odds. From this we derive per-player pre-match features.

Initial data understanding revealed most of the missing values concentrated in the number of games won in the n-th set column and in some of the betting odds columns. We removed some of such columns (W1-W5, L1-L5, total sets), along with columns that weren't consistently available throughout all the years (BFEW, BFEL, EXW, etc.).
Betting odds were excluded from model inputs because the objective was to evaluate predictive performance based on historical tennis features rather than market-derived expectations. This also allowed a clearer interpretation of the contribution of engineered variables such as ranking differences, recent form and surface-specific performance. We later reintroduce them as an external benchmark after all model development is finished.
We also corrected ambiguous column labeling, removing unnecessary spaces, and different representations of the same kind of information.
Walkovers and retirements were excluded because they don't represent normally completed matches and may be determined by events not captured by the available pre-match features. Furthermore, rows with fundamental information missing (players, ranking, date, points) were also excluded.
These changes reduced the dataset to a final of 26536 usable matches.

We transformed the original Winner/Loser representation into a neutral Player 1/Player 2 representation so that the position of the player does not reveal the result of a match. A randomized target variable was created, identifying whether Player 1 or 2 won, forcing the algorithm to learn genuine relationships rather than memorizing positional patterns.

Feature engineering

All features are computed using only information available strictly before the match date; for matches played on the same day, features are computed for all of them first, and match results are only added to each player's history afterward, preventing same-day leakage. We built seven feature families:

- **Experience** (e.g. Player1MatchesBefore, ExperienceDifference): total prior matches played by each player. A more experienced player is likely better calibrated to competitive pressure.
- **Overall win rate** (e.g. Player1WinRateBefore, WinRateDifference): each player's smoothed career win rate, using (wins + 1) / (matches + 2) rather than a raw ratio so that new players default to a neutral 0.5 instead of an extreme 0% or 100% after just one match.

- **Recent form** (e.g. Player1Recent5WinRate, Recent5WinRateDifference): win rate over each player's last 5 matches (via a fixed-size deque).
- **Surface specific performance** (e.g. Player1SurfaceWinRateBefore, SurfaceWinRateDifference): smoothed win rate restricted to matches on the current surface, since player strength varies substantially between clay, hard, and grass.
- **Head-to-head record** (e.g. H2HMatchesBefore, DifferenceH2HWinRateBefore): prior direct meetings between the two players.
- **Rest days** (e.g. DaysSinceLastMatchDifference + binary missing-indicators): days since each player's last match, as a proxy for fatigue/rust. Missing indicators flag first-time appearances rather than silently imputing a value that could bias the signal.
- **Ranking/points differential** (e.g. RankDifference, PointsDifference): difference in ATP rank and ranking points between the two players.

All rate-based features draw only from a running history table updated match-by-match, so no feature can see its own match's outcome.

Missing values

For numeric features we filled missing values with the median of that column. For categorical features we used the mode of each column, and transformed them using the fitted OneHotEncoder.

Temporal Cross-Validation

Since tennis matches are collected over several years, the **order of the observations is important**. We build temporal cross-validation folds to make the model evaluation more realistic and to avoid using information from the future when predicting the past. Moreover, while a single chronological split would be enough, it would evaluate the model on only one future period. Its performance could therefore depend heavily on the particular year chosen for validation.
Therefore, we construct multiple temporal folds:

Training period	Validation period
2015-2018	2019
2015-2019	2020
2015-2020	2021
2015-2021	2022

Each fold represents a different **past-to-future prediction scenario**. This allows us to evaluate the model across several years, measure the variability of its performance, and obtain a more reliable average estimate. The expanding training window also reflects how a real model would progressively gain access to more historical data over time.
The performance obtained across the folds is then averaged to obtain a more reliable estimate of the model's generalization ability and to select the best hyperparameters. The same temporal cross-validation procedure is applied consistently to all the models used in the project.

Model Selection

Six classification methods were selected for comparing predictive approaches in tennis match outcome prediction: Random Forest, Artificial Neural Network, k-Nearest Neighbours, AdaBoost, XGBoost, and LightGBM.

Random Forest was chosen because it handles non-linear relationships well and provides feature-importance estimates that are useful for interpretation.
Artificial Neural Network offers a different modelling approach by learning complex non-linear combinations of the available features without predefined rules. This makes them suitable for investigating whether interactions between player statistics can be captured more effectively by a flexible parametric model.
K-Nearest Neighbours was selected as an instance-based method that classifies a new match according to historically similar matches, providing a useful contrast and emphasizing local similarity.
AdaBoost was included as a boosting method that sequentially focuses on observations misclassified by previous learners. This is particularly interesting for tennis prediction because some matches may be harder to classify than others.
XGBoost and LightGBM were selected as modern gradient boosting implementations and as methods beyond scikit-learn. Both are well suited to tabular data and can model complex feature interactions. Their inclusion also allows us to compare two highly optimized boosting frameworks that use different tree-growth and training strategies.

For each model, we defined a small manual grid of candidate configurations and evaluated every configuration across the four temporal cross-validation folds, selecting the configuration with the highest mean fold accuracy.

Model	**Hyperparameters tuned**	**Final values**	**Mean CV Accuracy**
LightGBM (beyong sklearn)	num_leaves, n_estimators, learning_rate	leaves=7, n_estimators=200, lr=0.05	0.645
AdaBoost (sklearn)	max_leaf_nodes, n_estimators, learning_rate	leaf_nodes=4, n_estimators=100, lr=0.5	0.643
XGBoost	max_depth, n_estimators, learning_rate	depth=2, n_estimators=200, lr=0.05	0.643
Random Forest	n_estimators, max_depth, min_samples_leaf	n_estimators=200, depth=10, leaf=1	0.642
ANN	hidden layer size, activation, learning_rate, epochs	[16] units, ReLU, lr=0.001, 50 epochs	0.641
k-NN	n_neighbors, weights	k=151, uniform weights	0.637

Despite LighGBM and AdaBoost being in the first places with respect to accuracy, no model particularly dominates. The difference of accuracy is approximately only one point between each model with a range of 0.645-0.637.

Results and evaluation

Model	Accuracy	ROC-AUC	LogLoss
XGBoost	0.6509	0.7119	0.6179
AdaBoost	0.6480	0.7104	0.6249
Bookmaker	0.685	0.7529	0.5864
Baseline (better-ranked player)	0.6422	NaN	NaN

We compare our models' results with the baseline and the bookmaker odds: XGBoost emerges as the model with the highest value in most categories, with AdaBoost following behind. They perform better than the baseline, which predicts based on the better-ranked player.
However, the bookmaker results still outperform our chosen models.

Feature importance

We assessed feature relevance in two ways: comparing native importance across the four tree-based models (RF, AdaBoost, XGBoost, LightGBM), and computing permutation importance on the 2023 validation set across all six models, including ANN and k-NN. Since the native importance measures are defined differently across model types, their raw values are not directly comparable. We therefore normalized the importance values within each model and focused primarily on feature rankings.

Feature	Avg. Rank (4 tree models)	Avg. Rank (all 6 models)	Top-10 in how many models
PointsDifference	**1.00**	3.83	5 / 6
RankDifference	4.00	4.33	5 / 6
SurfaceWinRateDifference	3.00	6.33	5 / 6
WinRateDifference	4.25	19.17	2 / 6
DaysSinceLastMatchDifference	16.25	7.50	**6 / 6**
Player2Points	9.50	9.00	4 / 6

The four tree models agreed strongly on feature rankings (pairwise Spearman correlations 0.72–0.89). Four features ranked in every tree model's top 10: **PointsDifference** (AvgRank=1.0), **SurfaceWinRateDifference** (3.0), **RankDifference** (4.0), and **WinRateDifference** (4.25); consistently the strongest and most stable predictors. Among them, PointsDifference was particularly stable, ranking first in all four models.
With all six models the consensus shifted slightly: **PointsDifference**, **RankDifference**, and **SurfaceWinRateDifference** remained top-ranked (confirming the native-importance result), but **DaysSinceLastMatchDifference** emerged as the single most consistent feature, top-10 in 6/6 models despite ranking modestly under native importance. This suggests that the difference in recent match activity provides a relatively stable predictive signal across the different model families.
Overall PointsDifference, RankDifference, and SurfaceWinRateDifference are robust across both methods and all six models and therefore the clearest predictive signal in the dataset.

Error analysis

“All models correct” and “all models wrong” subsets were compared on the 2024–2025 test set. The comparison below is restricted to feature distributions available before the match.

Characteristics	All models correct	All models wrong
Number of matches	2617.000000	1176.000000
Median absolute rank difference	51.000000	43.000000
Median absolute points difference	1195.000000	696.500000
Median absolute win-rate difference	0.128176	0.096095
Median absolute surface win-rate difference	0.134615	0.104145
Ranking upset percentage	4.508980	93.877551

When the better-ranked player wins, the models get it right almost every time. When the worse-ranked player wins instead (an "upset"), the models get it wrong most of the time (93.877%); accuracy on upsets drops to roughly 11–28%, compared to 84–95% when there's no upset. This is the main thing going on: the models are basically betting on the higher-ranked/higher-points player, so whenever that player loses, the model is wrong almost by definition.

Close matchups are the hardest. When two players are close in ranking, accuracy drops to around 57–60%. When one player is much higher ranked than the other, accuracy climbs to about 70%. This makes sense: if the ranking gap is small, there isn't much signal to go on, and upsets are also more common in these matchups.

Surface doesn't matter much. We checked whether the models do worse on clay vs. grass vs. hard court, expecting maybe clay to be trickier. However, accuracy is about the same everywhere (63–65%).

Even "confident" models can make mistakes. When XGBoost was very confident (80%+) in its pick, it was right much more often (about 85%), but it still got 83 matches wrong even at that confidence level. Looking at those, they're almost all upsets: a lower-ranked or hot-streak player beating a top seed.

Conclusion

In conclusion, after comparing six different classification models, XGBoost achieved the best overall performance on the untouched 2024–2025 test set. Although this improves on the simple better-ranked-player baseline, the relatively small difference shows how difficult tennis matches are to predict, especially when players have similar rankings or when an upset occurs. Overall, the results show that historical performance, rankings, form and surface-related features contain useful predictive information, but they cannot fully capture the uncertainty involved in real match outcomes.

This project was developed by Harbas Amina, Miotto Elisa, Ponga Agnese.